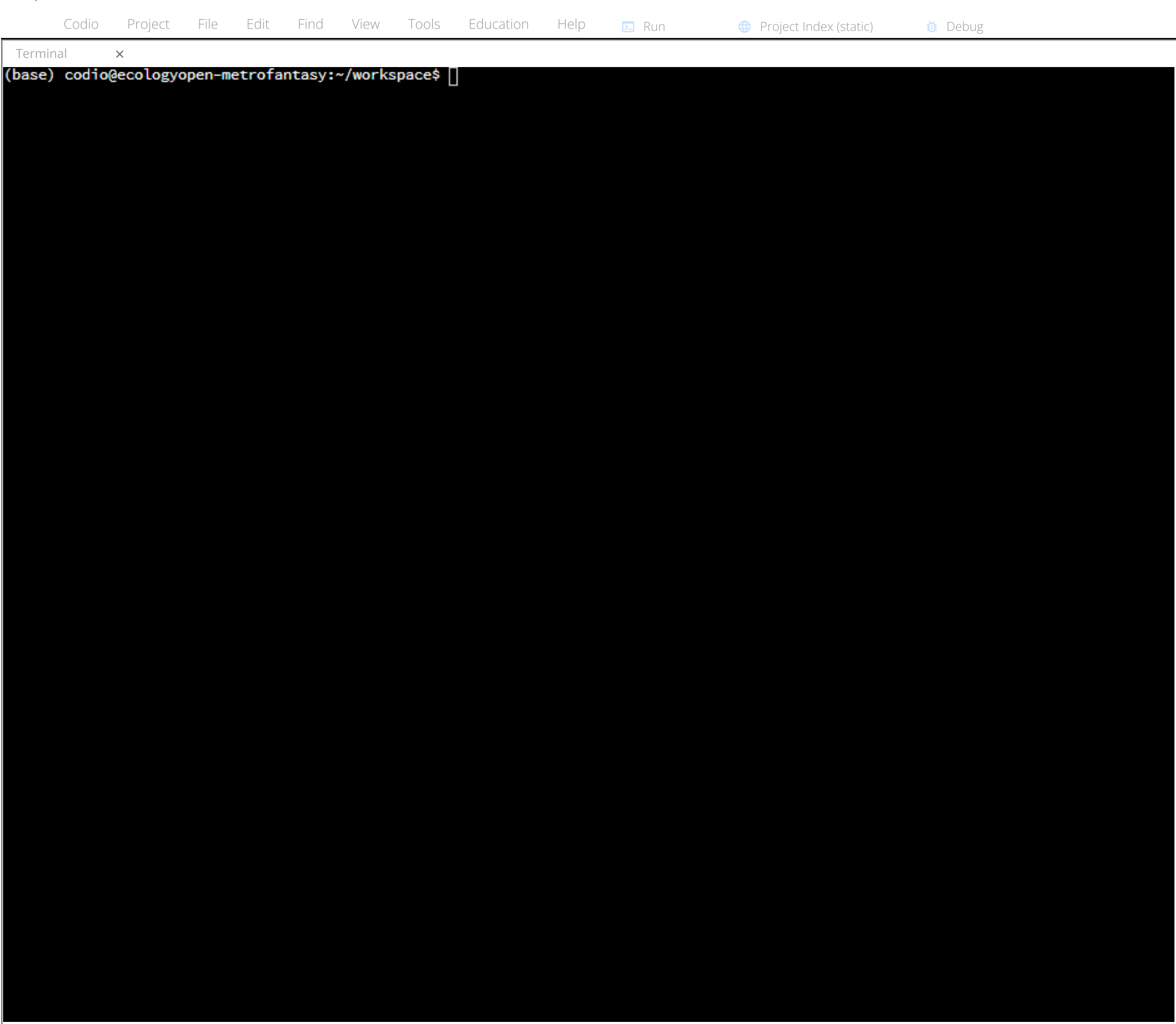




Code - Conda

Guide

Collapse

Requirements File

Creating and Using a Requirements File

Conda, like other package managers, allows you to generate an official list of all packages used in a particular virtual environment.

```
conda activate learning-conda
```

Use the `list` command followed by the `--explicit` flag to generate a list of installed packages. We do not want this list sent to the terminal, so use the `>` operator to redirect the list of packages to the file `requirements.txt`.

```
conda list --explicit > requirements.txt
```

Click the link below to open the `requirements.txt` file. This is the list of packages in the environment, and it can be used to generate copies of the environment.

Open requirements.txt

Notice toward the top of the requirements file, Conda tells you how to use this file to generate a new environment with the same packages. Here is the provided template:

```
# This file may be used to create an environment using:
# $ conda create --name <env> --file <this file>
# platform: linux-64
```

That means if we want to create the environment `new-env` that has the same packages as `learning-conda`, then we would use the following command:

```
conda create --name new-env --file requirements.txt
```

Packages Installed with `pip`

We know that using the `list` command produces a list of all of the packages installed. This includes packages installed with `pip`. Let's pipe list the packages and then pipe the result to the `grep` utility. We want to search for only those packages with `"python"` in the title.

```
conda list | grep "python"
```

There are two packages; Python 3.9 and `python-twitter`. **Note**, the version numbers are from the time of writing. Your version numbers may be different.

python	3.9.12	h12debd9_0	
python-twitter	3.5	pypi_0	pypi

The `--explicit` flag is supposed to create a list of packages for recreating an environment. Using `grep` again, search the explicit list of packages for those containing `"python"`.

```
conda list --explicit | grep "python"
```

This time, however, there is only one package — Python 3.9. There is no mention of `python-twitter`.

```
https://repo.anaconda.com/pkgs/main/linux-64/python-3.9.12-h12debd9_0.conda
```

We could use `pip` to create a requirements file. The `python-twitter` is included, but Python 3.9 is not listed in the output.

```
pip freeze
```

If your environment mixes packages from Anaconda and PyPI, the better option is to export the environment itself as `requirements.yaml`. Run the following command and click on the link below to open the newly created file.

```
conda env export > requirements.yaml
```

▼ **What is the `.yaml` file extension?**

`.yaml` is the file extension for YAML files. YAML stands for "Yet Another Markup Language" and was designed to be easy to read by humans. YAML is often used for configuration files.

Open requirements.yaml

If you scroll down a bit, you will see a section entitled `pip`. Here, you can find all of the dependencies installed via `pip`. Exporting the environment is the best way to generate a requirements file for your environment.

You can create a virtual environment from the YAML file by using `conda env create` as the command. Use the `--name` flag to give the new environment a name, and use the `--file` flag to indicate the name of the YAML file to be used.

```
conda env create --name from-yaml --file requirements.yaml
```

We can use `grep` to search the list of installed packages for those containing the phrase `"python"`. We should two results, Python 3.9 and the `python-twitter` package.

```
conda list --name from-yaml | grep "python"
```

Reading Question

Fill in the blanks below.

- `conda list --explicit` conda env export - creates a requirements file of all packages regardless of how they were installed.
- `conda env export` conda list --explicit - creates a requirements file of just those packages installed with Conda.
- pip freeze - creates a requirements file of just those packages installed with pip.

The correct answers are:

- conda env export** - creates a requirements file of all packages regardless of how they were installed.
- conda list --explicit** - creates a requirements file of just those packages installed with Conda.
- pip freeze** - creates a requirements file of just those packages installed with pip.

If you have a mixed environment with packages from both PyPI and Anaconda, the best way to generate a requirements file is to use `conda env export`. This includes packages from both repositories. Otherwise you risk ignoring packages.

Check ID

Next

<https://rodiu.com/sandpaw-dey/conda/terminal/backend-guide>

1/1